C and C++ Programming Languages Review

COMPE496: Introduction to Algorithms and Programming

> Comprehensive review of foundational programming concepts

STANDARD C

C Programming Language Fundamentals Define Modern Systems Programming

MAIN INSIGHT

C remains the foundation of systems programming due to its efficiency, low-level access, and widespread influence on modern languages.

Direct Memory Management: C provides precise control over system resources through pointers, arrays, and manual allocation/deallocation. It supports compound data types like structures and unions for efficient data organization.

Standard Libraries & Efficiency: Extensive libraries for I/O, math, and system operations offer comprehensive functionality with minimal overhead. The language has evolved from ANSI-C to C1X, adapting to new architectures.

Widespread Influence: C serves as the foundation for C++, C#, and Objective-C, and influenced Java, Python, and Perl. It powers operating systems, microcontrollers, drivers, and embedded systems where performance is critical.

DEV ENVIRONMENT

- **Compilers:** GCC, Clang, MSVC
- **IDEs:** VS Code, CLion, Vim
- **Debug:** GDB is essential

C Language Characteristics Reveal Trade-offs Between Power and Safety

Insight: C's low-level capabilities and efficiency come at the cost of safety features, requiring disciplined programming practices.

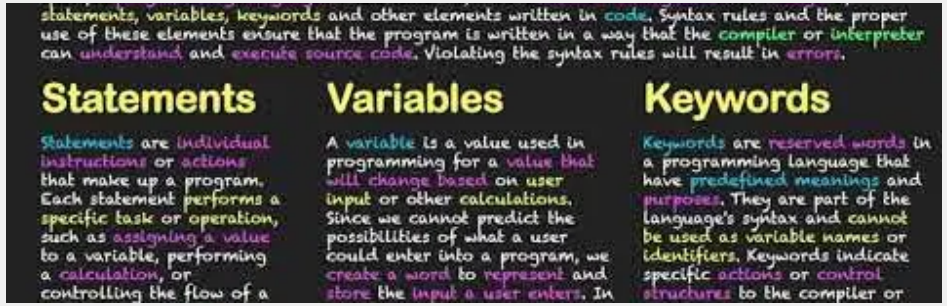
INHERENTLY UNSAFE POWER

C operates as a low-level language with direct hardware access. It lacks modern safety nets like exceptions, automatic range checking, and garbage collection.

> **Risk:** Type checking is limited to compile time. No runtime checks mean buffer overflows and memory errors are common if not managed carefully.

COMPILATION & STRUCTURE

Follows a direct **Edit** → **Compile** → **Link** → **Run** cycle. Source files (.c) and headers (.h) are manually managed. The preprocessor (#include) literally inserts file contents, requiring careful path management.



BASIC SYNTAX ELEMENTS

```
int n = 0;
```
Variable Declaration

```
Y = X + 5 / (Y * Y);
```
Arithmetic Expression

```
X += Y;
```
Compound Assignment

```
/* Comment */
```
Semicolon terminates statements

Operator Precedence and Expression Evaluation Require Careful Attention

Arithmetic expressions follow standard mathematical precedence. Unary operators are evaluated first, followed by multiplication/division, then addition/subtraction.

```
Y = X + 5 / (Y * Y);
// Parentheses override default precedence
```

Compound assignment operators (e.g., `X += Y`) provide shorthand for `X = X + Y` but have the lowest precedence.

BEST PRACTICE

Use parentheses to make the intended order explicit.

RANK	OPERATORS	DESCRIPTION	ASSOCIATIVITY
1 (High)	`+` `-`	Unary plus and minus (sign)	*Right-to-left*
2	`*` `/` `%`	Multiplication, division, modulus	*Left-to-right*
3	`+` `-`	Addition, subtraction	*Left-to-right*
4 (Low)	`=` `+=` `-=` `*=`	Assignment and compound assignment	*Right-to-left*

C++ Enhances C with Object-Oriented Capabilities While Maintaining Compatibility

MODERN C++ STANDARD

OBJECT-ORIENTED DESIGN

MAIN INSIGHT

C++ extends C with powerful abstractions while preserving backward compatibility, enabling gradual evolution of codebases.

ORIGINS & EVOLUTION

- Developed by **Bjarne Stroustrup** at Bell Labs as "C with Classes".
- **Superset of C:** The name "C++" implies the increment operator, signifying an enhanced version.
- **Compatibility:** C++ compilers can compile C programs, allowing incremental adoption.

"C++ is designed to be a statically typed, general-purpose, case-sensitive, free-form programming language that supports procedural, object-oriented, and generic programming."

KEY ENHANCEMENTS

- **Object-Oriented:** Introduces classes, encapsulation, inheritance, and polymorphism.
- **Stronger Typing:** Improves software quality and catches errors at compile time.
- **Advanced Features:**
  - Function & Operator Overloading
  - Templates for Generic Programming
  - Robust Standard Template Library (STL)

Object-Oriented Programming Focuses on Abstraction and Encapsulation

CORE PHILOSOPHY

OOP enables creating reusable tools with minimal implementation knowledge through data abstraction and information hiding.

01

DATA ABSTRACTION

Provides only essential information to the outside world, hiding background details and complexity.

EXAMPLE

A TV allows channel changes without requiring knowledge of its internal electronic circuitry.

02

INFORMATION HIDING

Restricts data access to authorized methods only. Separates the public interface from the private implementation.

EXAMPLE

Private class members are hidden from users but remain essential for internal functionality.

03

ENCAPSULATION

Bundles data with the methods that operate on it. Creates cohesive units that manage their own state.

BENEFIT

Reduces system complexity and improves maintainability by localizing changes.

Classes and Objects Form the Foundation of C++ Programming

Insight: Classes define types with both data and behavior, while objects instantiate these types as concrete entities.

The Class

A type definition that includes both data properties (state) and operations (behavior) permitted on that data.

The Object

A variable declared to be of some class. It is a concrete instance containing the specific data values.

> *"A variable is an instance of a primitive type. An object is an instance of a class."*

ACCESS CONTROL & INFORMATION HIDING

Public Members

Visible to all routines. Accessed by any method in any class. Typically used for **methods** (interface).

Private Members

Visible only to methods within the class. Hidden from non-class routines. Typically used for **data**.

C++ CLASS STRUCTURE

```
class IntegerCell {
  public:
    // Interface (Methods)
    int read();
    void write(int x);

  private:
    // Implementation (Data)
    int storedValue;
};
```

Constructors Initialize Objects and Establish Invariants

Insight: Constructors execute automatically when objects are created, ensuring proper initialization and establishing class invariants.

CORE MECHANICS

Automatic Execution: Runs immediately when an object is declared to set its initial state.

Identity: Must have the same name as the class and **no return type** (not even void).

Overloading: Multiple constructors can exist with different parameter lists.

Default Constructor: A constructor with no arguments. Automatically generated if no other constructor is defined.

INTEGERCELL.H

```
class IntegerCell {
public:
    // Default constructor
    IntegerCell() { storedValue = 0; }

    // Constructor with parameter
    IntegerCell(int initialValue) {
        storedValue = initialValue;
    }

    // Modern C++: Initializer List & Default Args
    IntegerCell(int initialValue = 0)
        : storedValue(initialValue) { }

private:
    int storedValue;
};
```

CORRECT DECLARATION

```
IntegerCell m1;
IntegerCell m2(12);
```

COMMON PITFALL

```
IntegerCell m4();
```

⚠ Declares a function, not an object!

Destructors Perform Cleanup When Objects Are Destroyed

Insight: Destructors ensure proper resource cleanup and prevent memory leaks by executing automatically when objects go out of scope.

Separating Interface from Implementation Enables Modular Design

Insight: Dividing class declarations from implementations improves maintainability, enables information hiding, and supports large-scale development.

The Interface (.h)

Contains declarations and function prototypes. It defines *what* the class does without revealing *how*. Stable and shared.

The Implementation (.cpp)

Contains the actual function definitions. Can be modified and recompiled independently without affecting code that uses the interface.

HEADER GUARDS

Prevent multiple inclusion errors during compilation using preprocessor directives (`#ifndef`, `#define`, `#endif`).

IntCell.h — INTERFACE

```
#ifndef _INTCELL_H_
#define _INTCELL_H_

class IntCell {
public:
    explicit IntCell(int
initialValue = 0);
    int read() const;
    void write(int x);
private:
    int storedValue;
};
#endif
```

IntCell.cpp — IMPLEMENTATION

```
#include "IntCell.h"

IntCell::IntCell(int initialValue)
: storedValue(initialValue) {}

int IntCell::read() const {
    return storedValue;
}

void IntCell::write(int x) {
    storedValue = x;
}
```

Accessor and Mutator Functions Control Object State

Insight: Distinguishing between accessors and mutators clarifies intent and enables `const`-correctness for safer code.

ACCESSOR

READ-ONLY

A method that examines but **does not change** the state of its object. It allows safe data retrieval.

```
int read() const;
```

MUTATOR

READ-WRITE

A method that **modifies** the state of an object. It changes internal data values.

```
void write(int x);
```

INTEGERCELL.H • USAGE

```
class IntegerCell {
public:
    // Accessor: Marked with const
    int read() const { return storedValue; }

    // Mutator: No const qualifier
    void write(int x) { storedValue = x; }

private:
    int storedValue;
};

IntegerCell m1;
m1.write(44);          // Mutator changes state
cout << m1.read();     // Accessor reads state

const IntegerCell m2;
// m2.write(5);        // ERROR: Cannot call mutator on const object
cout << m2.read();     // OK: read() is const
```

Function Overloading Enables Natural Syntax for Similar Operations

Insight: Function overloading allows multiple functions with the same name but different parameters, improving code readability and expressiveness.

The Concept

Functions can share the same name if they perform similar tasks on different data types. This eliminates the need for distinct names like `squareInt` or `squareFloat`.

How It Works

The compiler selects the correct function at compile-time based on the arguments provided in the call. This is known as **Type-Safe Linkage**.

THE FUNCTION SIGNATURE

Function Name + Parameter Types

(Return type is NOT part of the signature)

OVERLOADING_EXAMPLE.CPP

```cpp
// 1. Signature: square(int)
int square(int x) {
    return x * x;
}

// 2. Signature: square(double)
double square(double y) {
    return y * y;
}

int main() {
    // Calls int version
    cout << "Int: " << square(7) << endl;

    // Calls double version
    cout << "Double: " << square(7.5) << endl;

    return 0;
}
```

Operator Overloading Extends Natural Syntax to User-Defined Types

Insight: Operator overloading enables user-defined types to use familiar operators, making custom classes feel like built-in types.

CONCEPT

Allows standard operators (+, *, ==) to work with objects. The function name is the keyword `operator` followed by the symbol.

CONSTRAINTS

- × Cannot create **new** operators (e.g., **).
- × Cannot change behavior for **built-in** types (e.g., `int + int`).
- × Precedence and associativity remain fixed.

WITHOUT OVERLOADING

```
c3 = c1.multiply(c2);
```

WITH OVERLOADING

```
c3 = c1 * c2;
```

COMPLEX.CPP

```
class Complex {
public:
    // Overload the * operator
    Complex operator*(Complex rhs) const;
};

// Implementation
Complex Complex::operator*(Complex rhs) const {
    Complex prod;
    // (a+bi)(c+di) = (ac-bd) + (ad+bc)i
    prod.re = (re * rhs.re - im * rhs.im);
    prod.im = (re * rhs.im + im * rhs.re);
    return prod;
}
```

C++ Input/Output Streams Provide Type-Safe, Extensible I/O

Insight: C++ streams offer a clean, type-safe alternative to C's printf/scanf, with automatic type handling and extensibility.

CORE COMPONENTS

Objects: `cout` (standard output) and `cin` (standard input) are objects of class `ostream` and `istream`.

Operators: Uses Insertion (<<) for output and Extraction (>>) for input.

Manipulators: Functions like `fixed`, `setprecision`, and `setw` control formatting directly in the stream.

STREAM FORMATTING EXAMPLE

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double val = 34.3434343;

    // Type-safe output with manipulators
    cout << fixed << showpoint;
    cout << setprecision(2);

    cout << "Value: " << setw(10) << val << endl;

    return 0;
}
```

WHY IT'S BETTER THAN PRINTF

Programming Paradigms Shape How We Organize and Think About Code

Insight: A paradigm is not just a language feature; it is a mindset that dictates how we model problems and structure solutions.

> DEFINITION
> A Programming Paradigm is a fundamental style or approach to programming, serving as a blueprint for the "world view" of the code.

Imperative

"How to do it"

- Focuses on describing the **steps** to achieve a result.
- Uses statements that change a program's **state**.
- Explicit control flow (loops, conditionals).
- **Sub-paradigms:** Procedural, Object-Oriented.

EXAMPLES
C, C++, Java, Python

Declarative

"What to do"

- Focuses on describing the **desired result**.
- Abstracts away the control flow and state changes.
- Often relies on immutable data and expressions.
- **Sub-paradigms:** Functional, Logic, Database Query.

EXAMPLES
SQL, Haskell, Prolog, HTML

C Embodies the Imperative Paradigm with Explicit State Management

Insight: Imperative programming focuses on describing *how* a program operates by changing its state through a sequence of statements.

CORE CHARACTERISTICS

Mutable State:

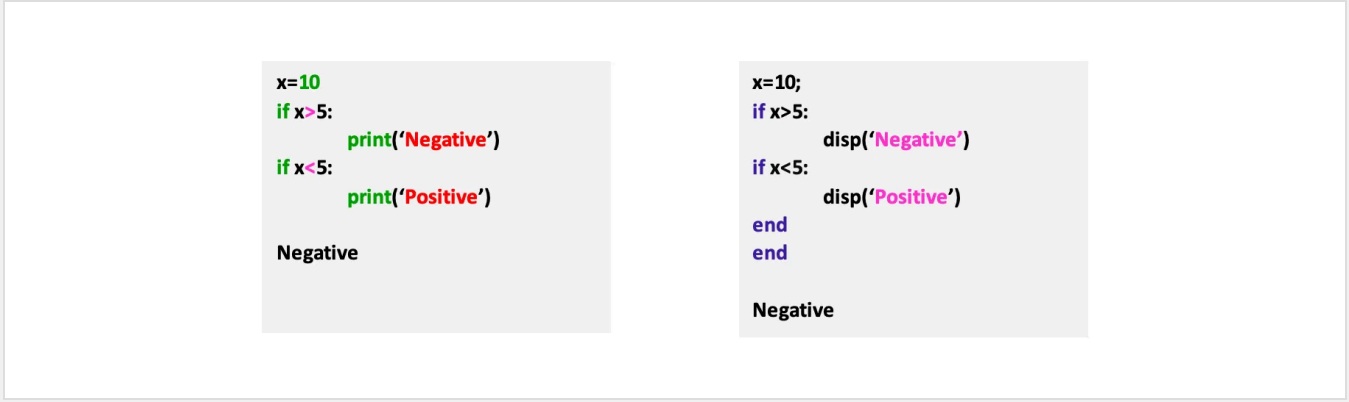
Variables are containers whose values change over time (assignment).

Control Flow:

Execution order is explicitly controlled via loops, conditionals, and function calls.

Von Neumann Roots:

Closely mirrors the underlying hardware architecture (fetching instructions, modifying memory).



IMPERATIVE LOGIC (C)

```c
// Goal: Calculate sum of 0 to 9
int total = 0;  // Explicit state initialization

// Explicit control flow (HOW to do it)
for (int i = 0; i < 10; i++) {
    total += i; // State mutation (Side effect)
}

// Result is stored in 'total'
```

C++ Extends C with Object-Oriented Paradigm Support

Insight: C++ transforms the imperative model by bundling data with behavior, turning passive structures into active objects.

IMPERATIVE (C)

Passive Data: Structs only hold data. Functions are external and operate *on* the data. Logic and state are separated.

OBJECT-ORIENTED (C++)

Active Objects: Classes encapsulate both data and the functions that manipulate it. Objects manage their own state.

C STYLE: SEPARATION

```
struct Rectangle { int width, height; }; // Function is separate from data
int area(struct Rectangle* r) { return r->width * r->height; }
```

↓

C++ STYLE: ENCAPSULATION

```
class Rectangle { int width, height; public: // Function is part of the
object int area() { return width * height; } };
```

Shift: "Doing things to data" → "Asking objects to do things"

Best Practices for C Programming Emphasize Safety and Clarity

Insight: C places the burden of safety on the programmer. Adopting strict discipline in memory and buffer management is essential to prevent critical vulnerabilities.

MEMORY SAFETY

- **Initialize Pointers:** Always set pointers to `NULL` after declaration and after freeing them to avoid dangling pointers.
- **Check Allocations:** Always verify that `malloc` returned a valid pointer before using it.
- **Pair Allocations:** Every `malloc` must have a corresponding `free`.

PATTERN

```
free(ptr);
ptr = NULL;
// Prevents double-free
```

BUFFER SECURITY

- **Avoid Unsafe Functions:** Never use `gets()`, `strcpy()`, or `sprintf()` without length limits.
- **Bounds Checking:** Always validate array indices against the size of the array.
- **Input Validation:** Sanitize all external inputs before processing.

COMPARISON

```
~~gets(buffer);~~
fgets(buf, size, stdin);
// Prevents overflow
```

TYPE CLARITY

- **Fixed-Width Integers:** Use `<stdint.h>` types like `int32_t` instead of vague types like `long` which vary by platform.
- **Const Correctness:** Use `const` for variables that should not change.
- **No Magic Numbers:** Use `#define` or `const` for array sizes.

MODERN C

```
~~unsigned long x;~~
uint64_t x;
// Explicit size
```

Best Practices for C++ Programming Leverage Modern Language Features

Insight: Modern C++ prioritizes safety and resource management through RAII, smart pointers, and strict const-correctness.

01

RAII PRINCIPLE

Resource Acquisition Is Initialization. Bind the life cycle of a resource (memory, file, lock) to the life cycle of an object. Resources are automatically released when the object goes out of scope.

EXAMPLE: AUTO-CLOSING FILE

```
void writeLog() {
  // File closes automatically
  std::ofstream file("log.txt");
  file << "Data";
} // Destructor called here
```

02

SMART POINTERS

Avoid raw pointers and manual `delete`. Use `std::unique_ptr` for exclusive ownership and `std::shared_ptr` for shared ownership to prevent memory leaks.

EXAMPLE: NO 'NEW' OR 'DELETE'

```
// Create unique pointer
auto ptr = std::make_unique<Widget>();

ptr->doSomething();
// Memory freed automatically
```

03

CONST CORRECTNESS

Mark variables and methods as `const` whenever possible. Pass large objects by `const reference` to avoid copying while ensuring immutability.

EXAMPLE: SAFE & EFFICIENT

```
// Read-only reference
void process(const Data& data) {
  // data.modify(); // Error!
  data.read(); // OK
}
```

C and C++ Continue to Power Critical Systems and Applications

Insight: Despite the rise of newer languages, C and C++ remain dominant in domains where performance, hardware control, and predictability are paramount.

SYSTEMS INFRASTRUCTURE

The backbone of modern computing. Code that runs close to the metal and manages hardware resources.

- →**Operating Systems (Linux Kernel, Windows, macOS)**
- →**Device Drivers & Firmware**
- →**File Systems & Virtualization Hypervisors**

EMBEDDED & REAL-TIME

Systems with strict constraints on memory, power, and timing. Reliability is critical.

- →**Automotive ECUs (Engine Control, Autonomous Driving)**
- →**Aerospace & Avionics Systems**
- →**Medical Devices & IoT Appliances**

HIGH-PERFORMANCE COMPUTING

Applications requiring massive computational throughput and low latency.

- →**High-Frequency Trading (HFT) Platforms**
- →**Scientific Simulations & Weather Forecasting**
- →**AI/ML Backends (TensorFlow, PyTorch Core)**

INTERACTIVE MEDIA

Real-time rendering and complex logic processing for entertainment and visualization.

- →**Game Engines (Unreal Engine, Unity, Godot)**
- →**Web Browser Engines (Chrome V8, WebKit)**
- →**Graphics Libraries (DirectX, OpenGL, Vulkan)**

Summary and Key Takeaways

Insight: Mastering C and C++ bridges the gap between hardware reality and software abstraction, forming the backbone of high-performance computing.

THE FOUNDATION (C)

Imperative Control

Explicit state management and control flow that closely mirrors hardware execution.

Memory Responsibility

Manual allocation and deallocation require disciplined pointer usage to prevent leaks.

Simplicity

A small, efficient standard library focused on performance and portability.

LEGACY & SYSTEMS

THE EVOLUTION (C++)

Object-Oriented

Encapsulation, inheritance, and polymorphism enable modular, reusable software design.

Abstraction

Classes, templates, and operator overloading allow creating custom types that behave naturally.

Modern Features

Stronger type checking and standard libraries (STL) enhance productivity.

ARCHITECTURE & SCALE

THE DISCIPLINE

Safety First

Use RAII, smart pointers, and const-correctness to eliminate common errors.

Design Separation

Strictly separate interface (.h) from implementation (.cpp) for maintainability.

Defensive Coding

Always validate inputs, check bounds, and handle resources explicitly.

PROFESSIONAL PRACTICE